

DESAIN ENDPOINT DAN DOKUMENTASI API WISATAKU

Nama : Ahmad Damanhuri
NIM : SI21240001
Program Studi : Sistem Informasi
Mata Kuliah : Teknologi Web Service

BAB I

PENDAHULUAN

1.1 Latar Belakang

Web Service merupakan teknologi yang memungkinkan aplikasi untuk saling berkomunikasi melalui jaringan menggunakan protokol dan format data tertentu. Salah satu pendekatan yang banyak digunakan dalam pengembangan Web Service adalah REST API.

Pada Praktikum ini, dilakukan perancangan endpoint API untuk aplikasi WisataKu. Aplikasi WisataKu merupakan aplikasi yang digunakan untuk menyediakan informasi mengenai destinasi wisata dan ulasan dari pengguna.

Pada Praktikum ini, API dirancang menggunakan NestJS dengan pendekatan REST. Selain membuat endpoint, API juga dilengkapi dengan DTO, validasi data, serta dokumentasi menggunakan OpenAPI/Swagger.

Tahap ini masih berfokus pada desain dan dokumentasi API sehingga response yang digunakan masih berupa data dummy dan belum terhubung dengan database Prisma dan MariaDB.

1.2 Tujuan

Tujuan dari Praktikum adalah:

1. Merancang endpoint REST API untuk aplikasi WisataKu.
2. Menentukan HTTP Method yang sesuai untuk setiap endpoint.
3. Menentukan status code yang digunakan oleh API.
4. Membuat DTO untuk data yang diterima oleh API.
5. Menerapkan validasi menggunakan class-validator.
6. Mengintegrasikan Swagger untuk dokumentasi API.
7. Menghasilkan dokumentasi API dalam format OpenAPI JSON.
8. Menguji endpoint melalui Swagger dan HTTP request.

BAB II

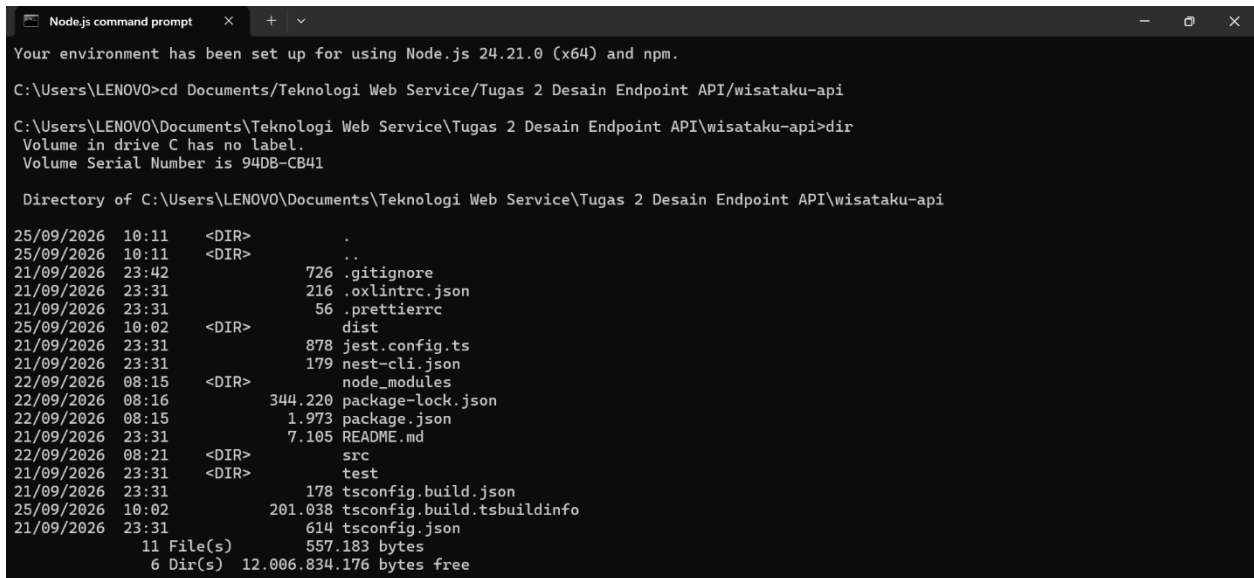
PERSIAPAN PROJECT

2.1 Membuka Project

Project dibuat menggunakan NestJS dengan nama project:

wisataku-api

Project berada pada folder:



```
Node.js command prompt
Your environment has been set up for using Node.js 24.21.0 (x64) and npm.
C:\Users\LENOVO>cd Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api
C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>dir
Volume in drive C has no label.
Volume Serial Number is 94DB-CB41

Directory of C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api

25/09/2026 10:11 <DIR>      .
25/09/2026 10:11 <DIR>      ..
21/09/2026 23:42          726 .gitignore
21/09/2026 23:31          216 .oxlintrc.json
21/09/2026 23:31           56 .prettierrc
25/09/2026 10:02 <DIR>      dist
21/09/2026 23:31          878 jest.config.ts
21/09/2026 23:31          179 nest-cli.json
22/09/2026 08:15 <DIR>      node_modules
22/09/2026 08:16       344.220 package-lock.json
22/09/2026 08:15          1.973 package.json
21/09/2026 23:31          7.105 README.md
22/09/2026 08:21 <DIR>      src
21/09/2026 23:31 <DIR>      test
21/09/2026 23:31          178 tsconfig.build.json
25/09/2026 10:02       201.038 tsconfig.build.tsbuildinfo
21/09/2026 23:31          614 tsconfig.json
               11 File(s)          557.183 bytes
               6 Dir(s)       12.006.834.176 bytes free
```

gambar 1. Struktur folder project WisataKu

Project ini digunakan sebagai source code untuk membangun API WisataKu.

2.2 Menginstal Package yang Dibutuhkan

Beberapa package tambahan diperlukan, yaitu:

- class-validator
- class-transformer
- @nestjs/swagger
- swagger-ui-express

Package tersebut digunakan untuk melakukan validasi data dan membuat dokumentasi API menggunakan Swagger.

Perintah yang digunakan:



```
C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>npm.cmd install class-validator class-transf
ormer @nestjs/swagger swagger-ui-express

changed 5 packages, and audited 603 packages in 50s

131 packages are looking for funding
  run 'npm fund' for details

5 vulnerabilities (2 low, 1 moderate, 2 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run 'npm audit' for details.
npm warn install-scripts 3 packages have install scripts not yet covered by allowScripts:
npm warn install-scripts @parcel/watcher@2.6.0 (install: node-gyp rebuild)
npm warn install-scripts @scarf/scarf@1.4.0 (postinstall: node ./report.js)
npm warn install-scripts unrs-resolver@1.12.2 (postinstall: node postinstall.js)
npm warn install-scripts
npm warn install-scripts Run 'npm install-scripts ls' to review, or 'npm install-scripts approve <pkg>' to allow.
C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>
```

gambar 2 Proses instalasi package yang dibutuhkan untuk validasi dan dokumentasi API

BAB III

PERANCANGAN ENDPOINT API

3.1 Perancangan Endpoint

Endpoint API WisataKu dirancang menggunakan prinsip REST. Endpoint yang dibuat terdiri dari enam endpoint utama.

No	Method	Endpoint	Fungsi
1	GET	/destinasi	Mendapatkan daftar destinasi
2	GET	/destinasi/:id	Mendapatkan detail destinasi
3	POST	/destinasi	Menambahkan destinasi
4	PATCH	/destinasi/:id	Memperbarui data destinasi
5	DELETE	/destinasi/:id	Menghapus destinasi
6	GET	/destinasi/:id/ulasan	Mendapatkan ulasan destinasi

Endpoint tersebut dibuat pada file:

src/destinasi/destinasi.controller.ts

```
import {
  Body,
  Controller,
  Get,
  Post,
  Param,
  Patch,
  Delete,
} from '@nestjs/common';

import {
  ApiTags,
  ApiOperation,
  ApiResponse,
} from '@nestjs/swagger';

import { CreateDestinasiDto } from '../dto/create-destinasi.dto';
import { UpdateDestinasiDto } from '../dto/update-destinasi.dto';

@ApiTags('Destinasi')
@Controller('destinasi')
export class DestinasiController {
  @Get()
  @ApiOperation({ summary: 'Mendapatkan semua destinasi wisata' })
  @ApiResponse({
    status: 200,
    description: 'Daftar destinasi berhasil diambil',
  })
  getAllDestinasi() {
    return [
      {
        id: 1,
        nama: 'Pantai Kuta Mandalika',
        kategori: 'Pantai',
        hargaTiket: 10000,
      },
    ],
  }
}
```

```

    ];
}

@Get('/:id')
@ApiOperation({ summary: 'Mendapatkan detail destinasi berdasarkan ID' })
@ApiResponse({
    status: 200,
    description: 'Detail destinasi berhasil diambil',
})
@ApiResponse({
    status: 404,
    description: 'Destinasi tidak ditemukan',
})
getDestinasiById(@Param('id') id: string) {
    return {
        id: Number(id),
        nama: 'Pantai Kuta Mandalika',
        kategori: 'Pantai',
        hargaTiket: 10000,
    };
}

@Post()
@ApiOperation({ summary: 'Menambahkan destinasi wisata baru' })
@ApiResponse({
    status: 201,
    description: 'Destinasi berhasil ditambahkan',
})
@ApiResponse({
    status: 400,
    description: 'Data yang dikirim tidak valid',
})
createDestinasi(@Body() createDestinasiDto: CreateDestinasiDto) {
    return {
        message: 'Destinasi berhasil ditambahkan',
        data: createDestinasiDto,
    };
}

@Patch('/:id')
@ApiOperation({ summary: 'Memperbarui data destinasi berdasarkan ID' })
@ApiResponse({
    status: 200,
    description: 'Destinasi berhasil diperbarui',
})
@ApiResponse({
    status: 404,
    description: 'Destinasi tidak ditemukan',
})
updateDestinasi(
    @Param('id') id: string,
    @Body() updateDestinasiDto: UpdateDestinasiDto,
) {
    return {
        message: `Destinasi dengan id ${id} berhasil diperbarui`,
        data: updateDestinasiDto,
    };
}

```

```

@Delete('/:id')
@ApiOperation({ summary: 'Menghapus destinasi berdasarkan ID' })
@ApiResponse({
  status: 200,
  description: 'Destinasi berhasil dihapus',
})
@ApiResponse({
  status: 404,
  description: 'Destinasi tidak ditemukan',
})
deleteDestinasi(@Param('id') id: string) {
  return {
    message: `Destinasi dengan id ${id} berhasil dihapus`,
  };
}

@Get('/:id/ulasan')
@ApiOperation({ summary: 'Mendapatkan ulasan berdasarkan ID destinasi' })
@ApiResponse({
  status: 200,
  description: 'Daftar ulasan destinasi berhasil diambil',
})
@ApiResponse({
  status: 404,
  description: 'Destinasi tidak ditemukan',
})
getUlasanDestinasi(@Param('id') id: string) {
  return [
    {
      id: 1,
      destinasiId: Number(id),
      namaPengguna: 'Ahmad',
      rating: 5,
      komentar: 'Destinasi sangat bagus dan nyaman untuk dikunjungi.',
    },
  ];
}
}

```

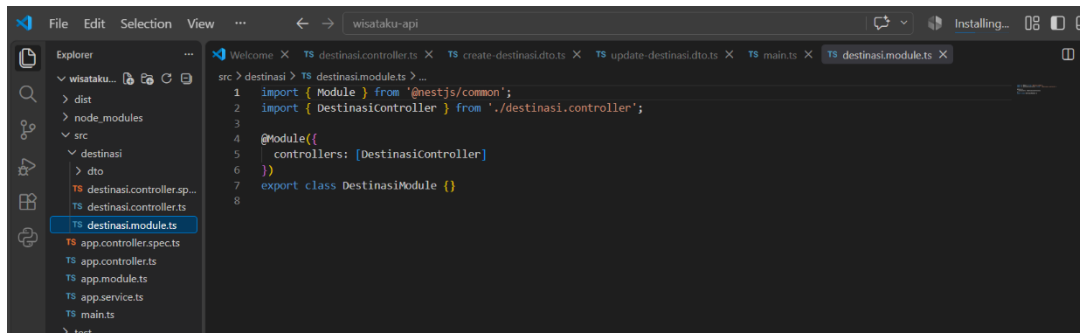
3.2 Pembuatan Module Destinasi

Untuk memisahkan fitur destinasi dari aplikasi utama, dibuat module destinasi.

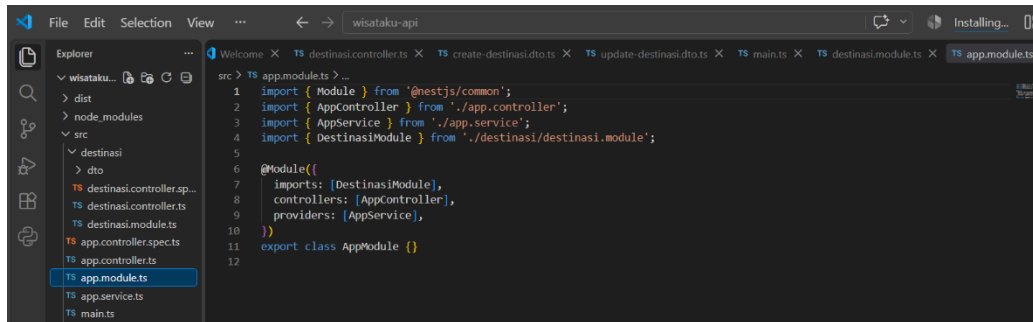
File yang digunakan:

src/destinasi/destinasi.module.ts

Module tersebut kemudian didaftarkan ke AppModule.



gambar 3 Implementasi DestinasiModule pada project WisataKu



gambar 4 Pendaftaran DestinasiModule pada AppModule

BAB IV

PEMBUATAN DTO DAN VALIDASI

4.1 CreateDestinasiDto

DTO digunakan untuk menentukan struktur data yang diterima ketika menambahkan destinasi baru.

File:

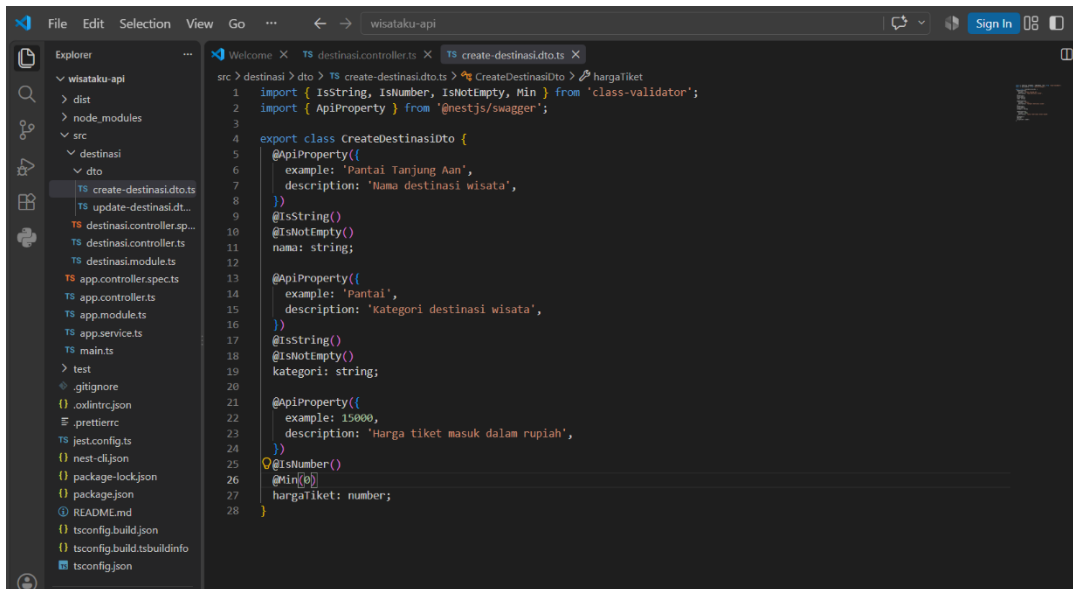
src/destinasi/dto/create-destinasi.dto.ts

DTO memiliki tiga field:

- nama
- kategori
- hargaTiket

Validasi yang diterapkan adalah:

- nama harus berupa string dan tidak boleh kosong.
- kategori harus berupa string dan tidak boleh kosong.
- hargaTiket harus berupa angka dan tidak boleh kurang dari 0.



gambar 5 Implementasi CreateDestinasiDto dan aturan validasi data

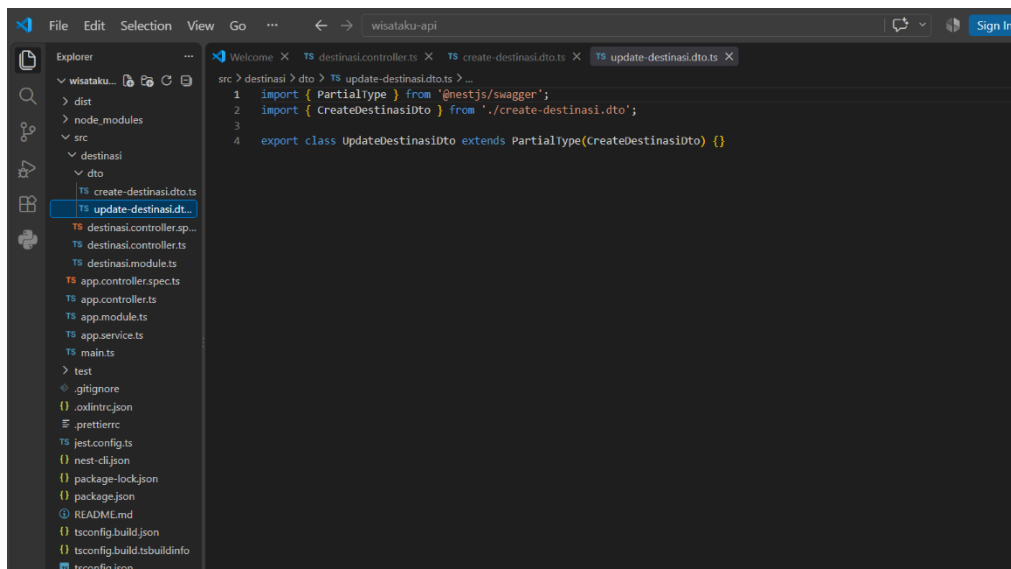
4.2 UpdateDestinasiDto

Untuk proses perubahan data destinasi digunakan UpdateDestinasiDto.

File:

src/destinasi/dto/update-destinasi.dto.ts

DTO ini menggunakan PartialType() dari Swagger sehingga seluruh field pada CreateDestinasiDto dapat digunakan sebagai field opsional ketika melakukan update.



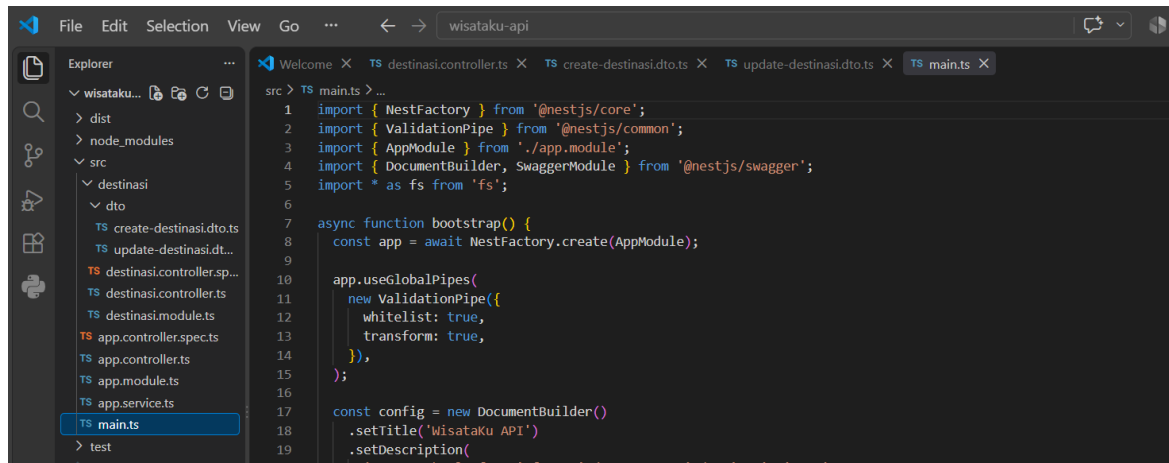
gambar 6 Implementasi UpdateDestinasiDto menggunakan PartialType

4.3 ValidationPipe

Validasi data diaktifkan secara global pada file:

src/main.ts

Konfigurasi yang digunakan:



gambar 7 Konfigurasi ValidationPipe pada src/main.ts

ValidationPipe digunakan agar data yang dikirim melalui request dapat diperiksa berdasarkan aturan validasi yang telah dibuat pada DTO.

BAB V

IMPLEMENTASI SWAGGER

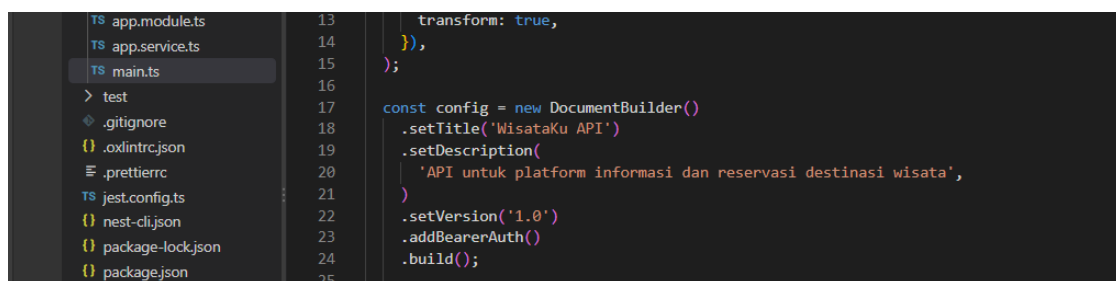
5.1 Konfigurasi Swagger

Swagger digunakan untuk membuat dokumentasi API secara otomatis berdasarkan controller, endpoint, DTO, dan decorator yang digunakan pada project.

Konfigurasi Swagger dilakukan pada:

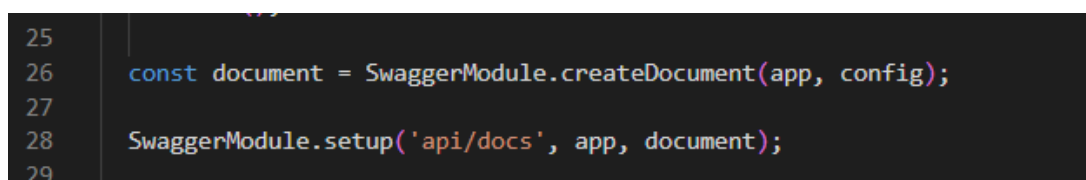
src/main.ts

Konfigurasi yang digunakan meliputi:



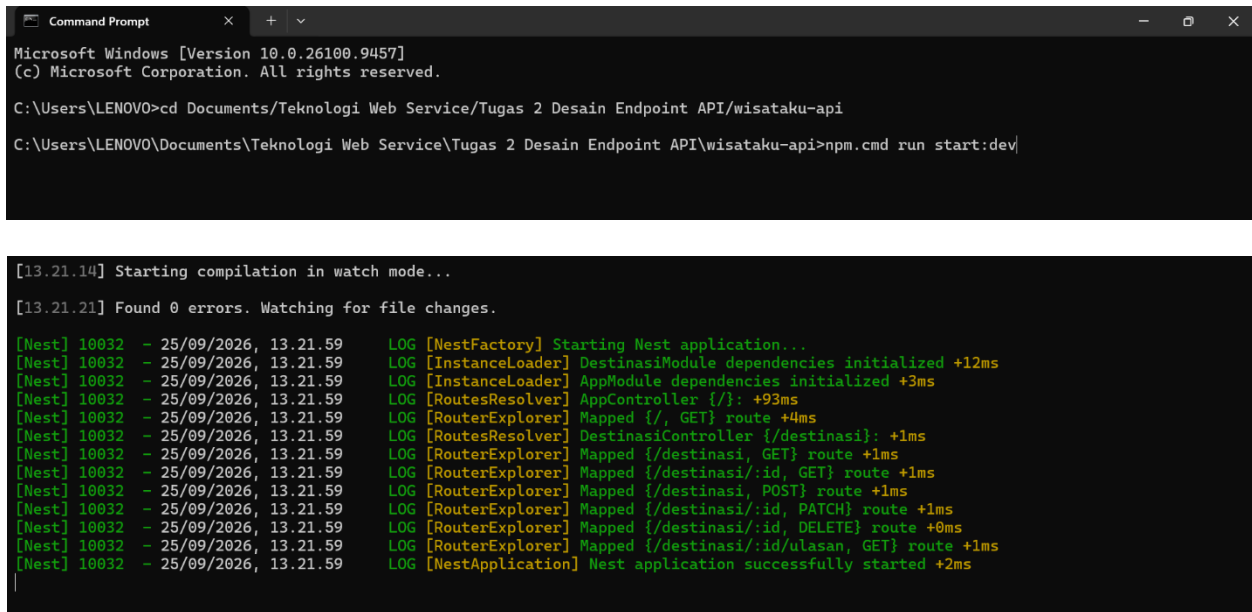
gambar 8 Konfigurasi Swagger pada src/main.ts

Kemudian dokumentasi Swagger dibuat menggunakan:



5.2 Menjalankan Project

Setelah konfigurasi selesai, project dijalankan menggunakan perintah:



```
Microsoft Windows [Version 10.0.26100.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\LENOVO>cd Documents/Teknologi Web Service/Tugas 2 Desain Endpoint API/wisataku-api

C:\Users\LENOVO\Documents\Teknologi Web Service\Tugas 2 Desain Endpoint API\wisataku-api>npm.cmd run start:dev

[13.21.14] Starting compilation in watch mode...
[13.21.21] Found 0 errors. Watching for file changes.

[Nest] 10032 - 25/09/2026, 13.21.59 LOG [NestFactory] Starting Nest application...
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [InstanceLoader] DestinasiModule dependencies initialized +12ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [InstanceLoader] AppModule dependencies initialized +3ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RoutesResolver] ApplicationController {/}: +93ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/, GET} route +4ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RoutesResolver] DestinasiController {/destinasi}: +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi, GET} route +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi/:id, GET} route +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi, POST} route +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi/:id, PATCH} route +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi/:id, DELETE} route +0ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [RouterExplorer] Mapped {/destinasi/:id/ulasan, GET} route +1ms
[Nest] 10032 - 25/09/2026, 13.21.59 LOG [NestApplication] Nest application successfully started +2ms
```

gambar 9 Proses menjalankan project menggunakan NestJS

Setelah project berhasil dijalankan, server akan berjalan pada:

<http://localhost:3000>

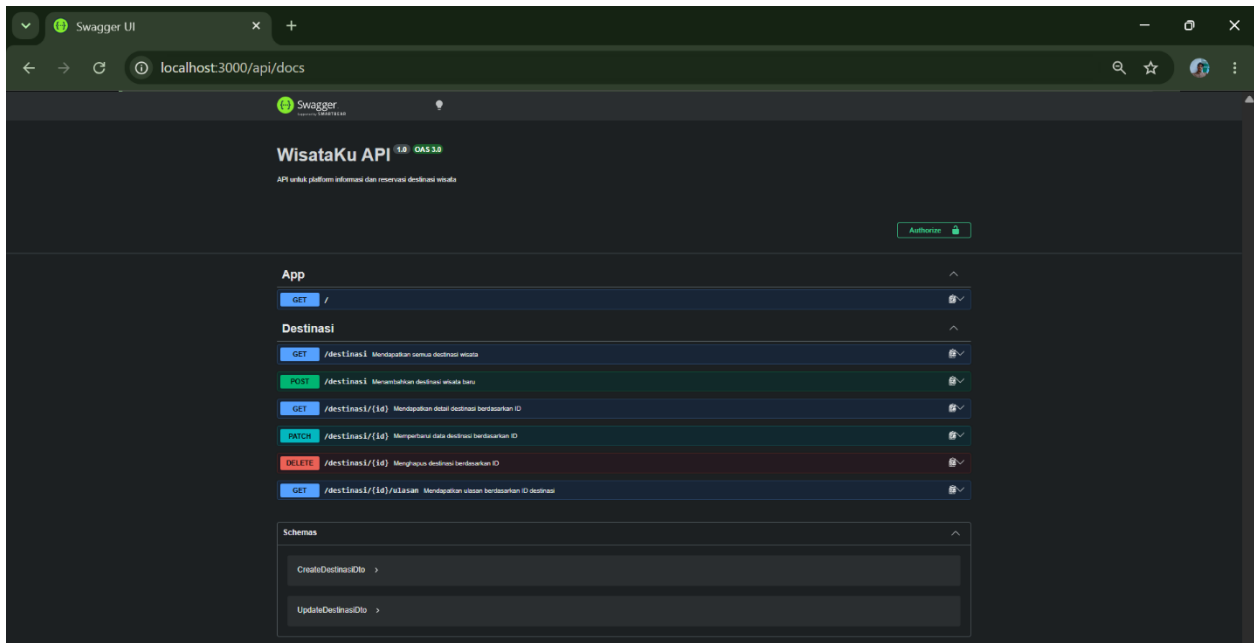


gambar 10. Hasil menjalankan http://localhost:3000

5.3 Membuka Swagger UI

Swagger UI dapat diakses melalui alamat:

<http://localhost:3000/api/docs>



gambar 11 Tampilan Swagger UI API WisataKu

Pada halaman Swagger UI terdapat dokumentasi API WisataKu.

Swagger menampilkan enam endpoint utama:

1. GET /destinasi
2. POST /destinasi
3. GET /destinasi/{id}
4. PATCH /destinasi/{id}
5. DELETE /destinasi/{id}
6. GET /destinasi/{id}/ulasan

Selain endpoint, Swagger juga menampilkan schema:

- CreateDestinasiDto
- UpdateDestinasiDto

BAB VI

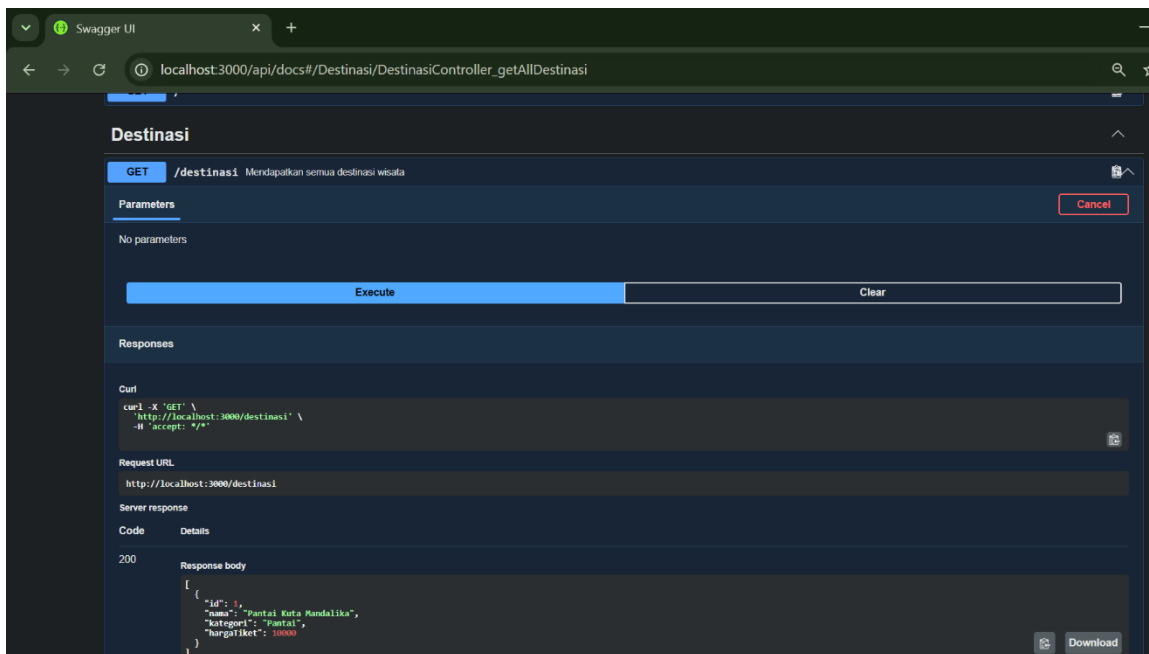
PENGUJIAN API

6.1 Pengujian GET /destinasi

Endpoint:

GET /destinasi

digunakan untuk mendapatkan daftar destinasi wisata.



gambar 12 Hasil pengujian GET /destinasi

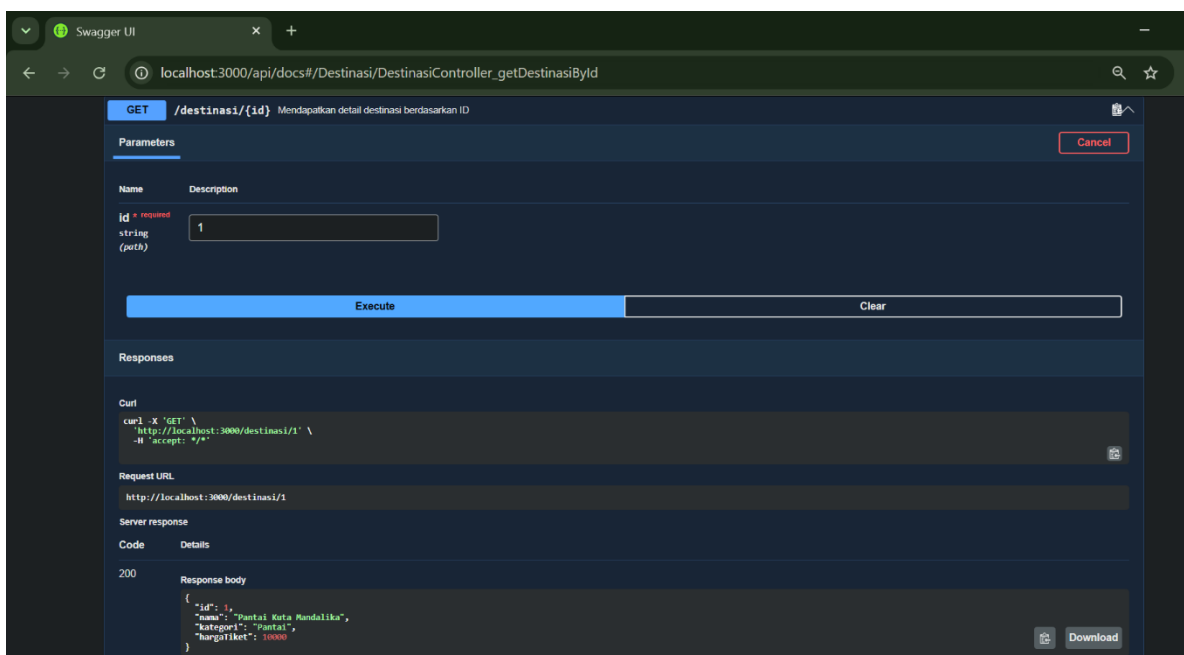
Hasil pengujian menunjukkan API berhasil mengembalikan data destinasi.

6.2 Pengujian GET /destinasi/:id

Endpoint:

GET /destinasi/1

digunakan untuk mendapatkan detail destinasi berdasarkan ID.



gambar 13 Hasil pengujian GET /destinasi/1

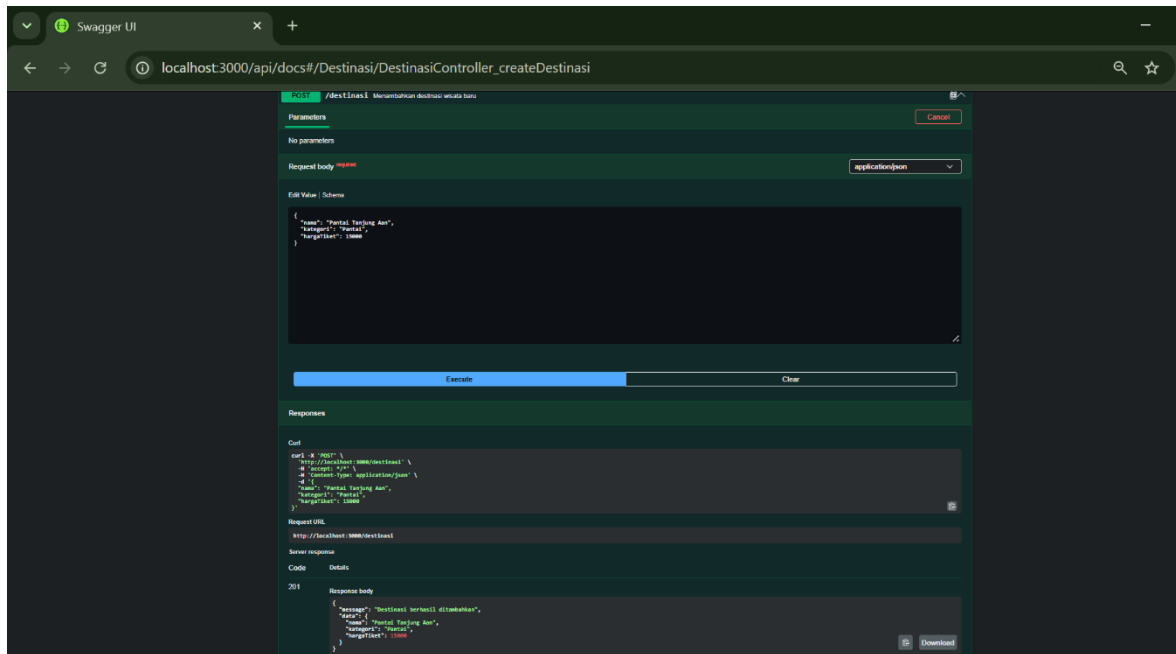
6.3 Pengujian POST /destinasi

Endpoint:

POST /destinasi

digunakan untuk menambahkan destinasi baru.

Data yang digunakan:



gambar 14 Hasil pengujian POST /destinasi dengan request body dan response

API berhasil menerima data dan mengembalikan response bahwa destinasi berhasil ditambahkan.

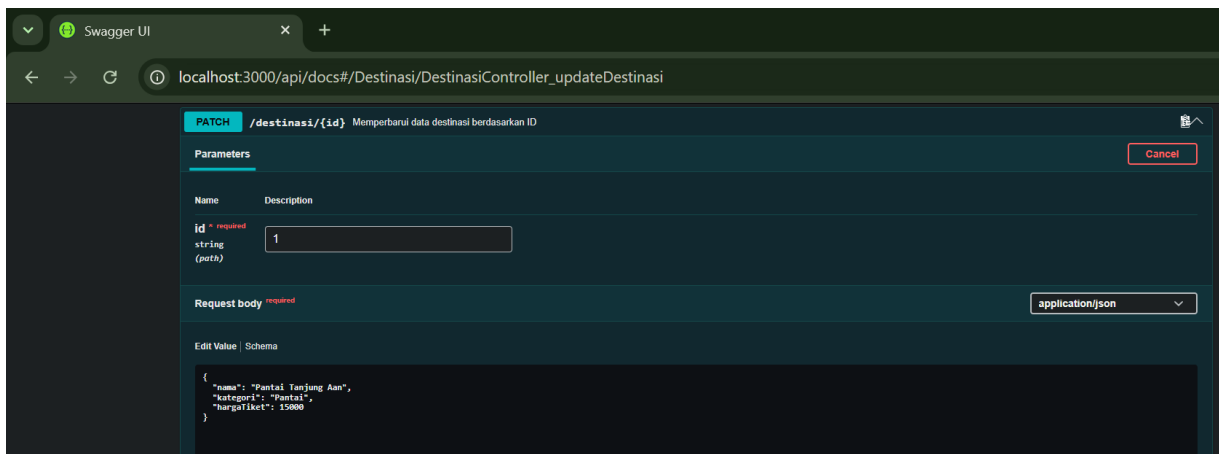
6.4 Pengujian PATCH /destinasi/:id

Endpoint:

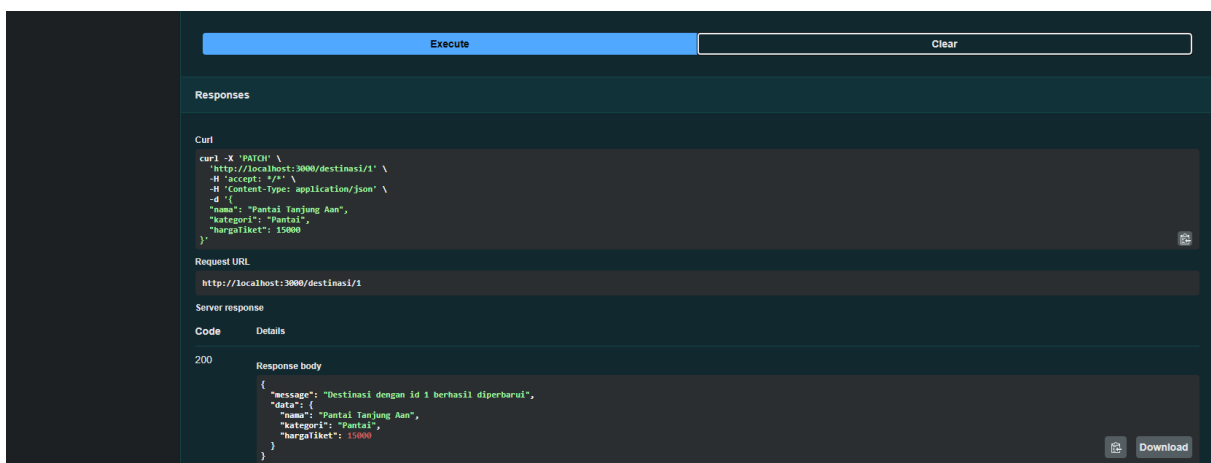
PATCH /destinasi/1

digunakan untuk memperbarui sebagian data destinasi.

Data yang digunakan:



gambar 15 pengujian PATCH /destinasi/1



gambar 16 Hasil pengujian PATCH /destinasi/1

```
{
  "nama": "Pantai Tanjung Aan",
  "kategori": "Pantai",
  "hargaTiket": 15000
}
```

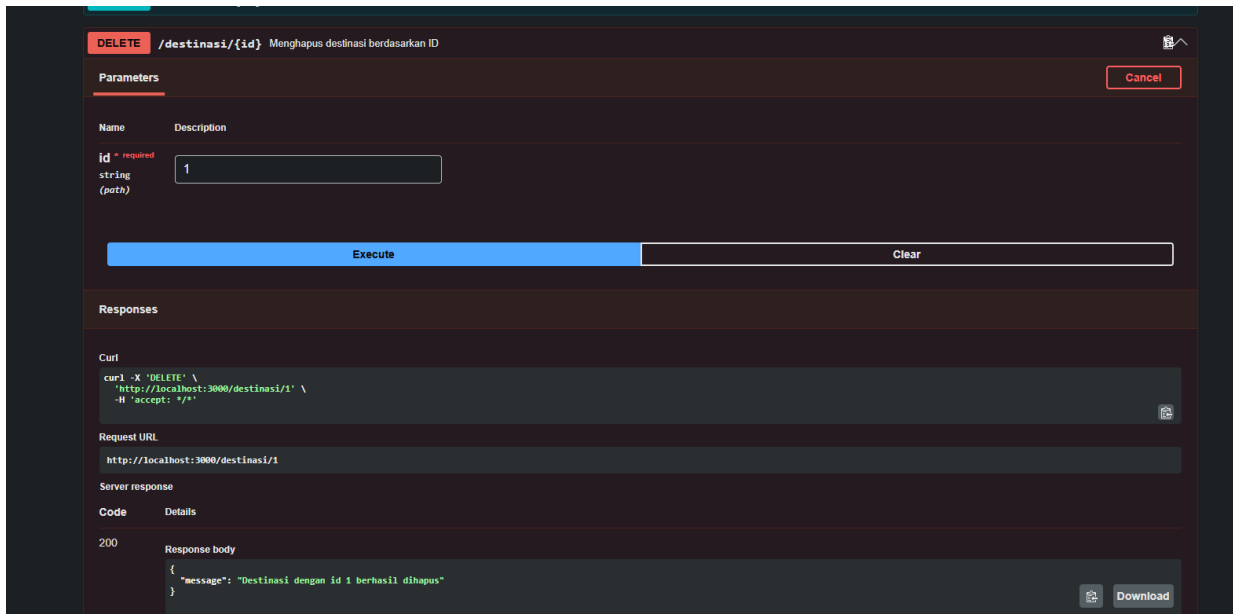
API berhasil menerima request dan mengembalikan response bahwa data destinasi berhasil diperbarui.

6.5 Pengujian DELETE /destinasi/:id

Endpoint:

DELETE /destinasi/1

digunakan untuk menghapus destinasi berdasarkan ID.



gambar 17 Hasil pengujian DELETE /destinasi/1

API berhasil mengembalikan response bahwa destinasi dengan ID tersebut berhasil dihapus.

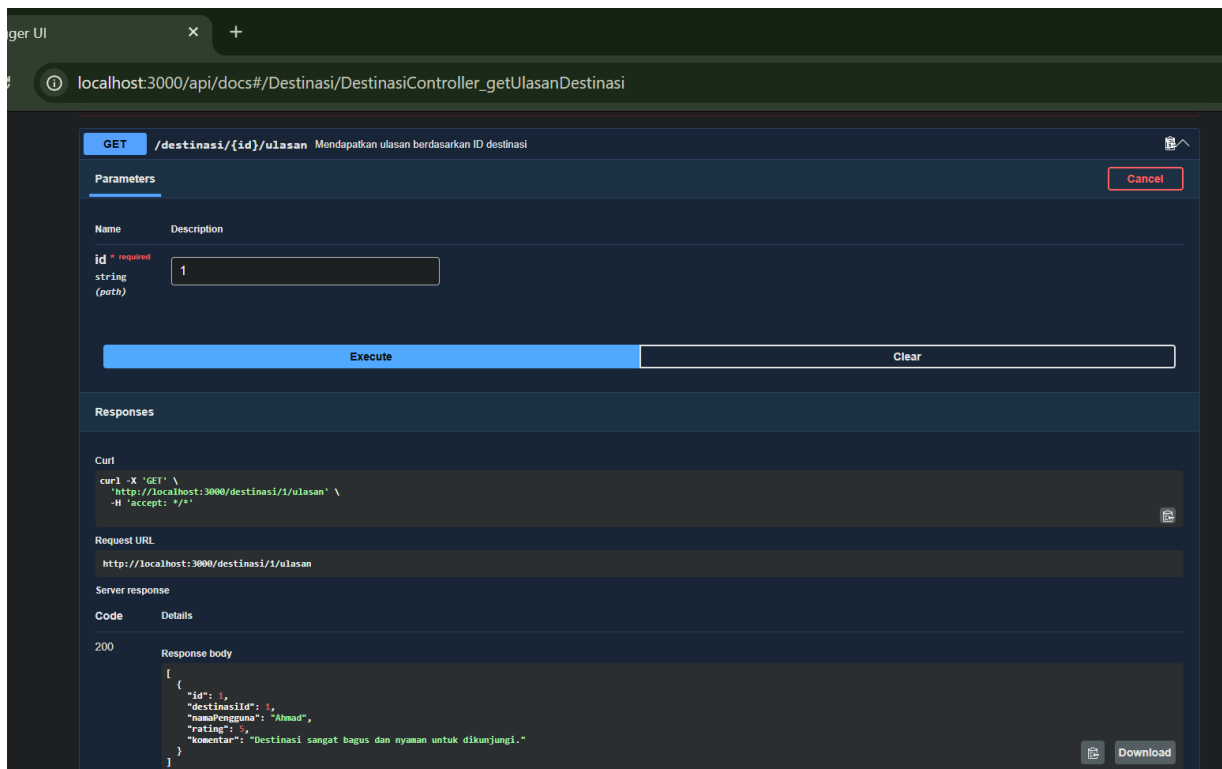
6.6 Pengujian GET /destinasi/:id/ulasan

Endpoint:

GET /destinasi/1/ulasan

digunakan untuk mendapatkan daftar ulasan berdasarkan ID destinasi.

API berhasil mengembalikan data ulasan.



gambar 18 Hasil pengujian GET /destinasi/1/ulasan

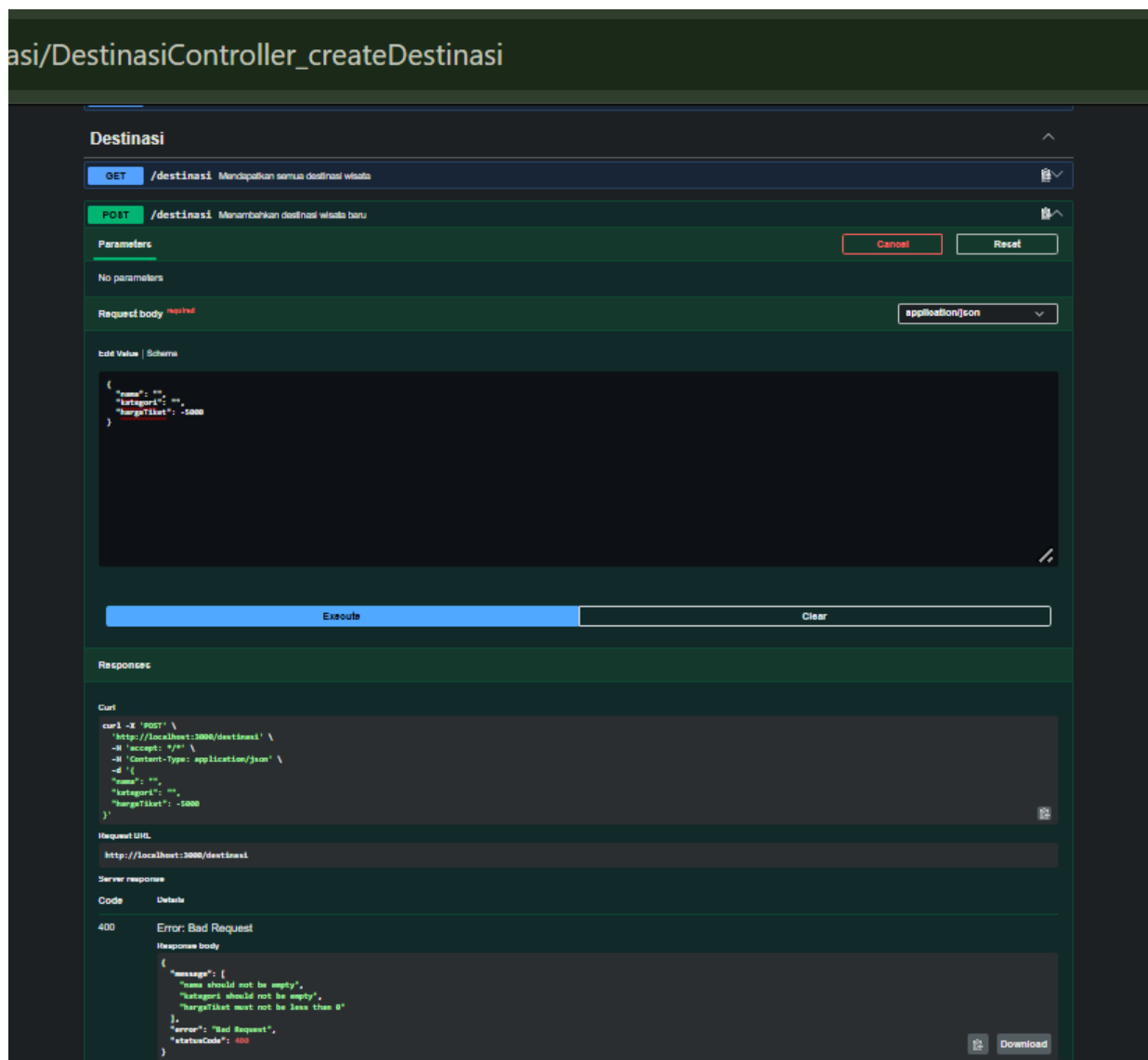
BAB VII

PENGUJIAN VALIDASI DATA

7.1 Pengujian Data Tidak Valid

Untuk menguji validasi, digunakan data:

```
{  
  
  "nama": "",  
  
  "kategori": "",  
  
  "hargaTiket": -5000  
}
```



gambar 19 Hasil pengujian data tidak valid dengan HTTP 400 Bad Request

Data tersebut tidak memenuhi aturan DTO karena:

- nama kosong.
- kategori kosong.
- hargaTiket bernilai negatif.

API kemudian mengembalikan:

400 Bad Request

dengan pesan validasi.

BAB VIII

PEMBUATAN OPENAPI.JSON

8.1 Export Dokumentasi OpenAPI

Dokumentasi Swagger kemudian diekspor menjadi file:

openapi.json

Proses export dilakukan pada file:

src/main.ts

dengan kode:

```
const document = SwaggerModule.createDocument(app, config);

SwaggerModule.setup('api/docs', app, document);

fs.writeFileSync(
  './openapi.json',
  JSON.stringify(document, null, 2),
);

await app.listen(process.env.PORT ?? 3000);
```

gambar 20 Implementasi kode export dokumentasi Swagger ke file openapi.json

File openapi.json berisi informasi mengenai:

- Endpoint API.
- HTTP Method.
- Parameter.
- Request Body.
- Response.
- DTO/schema.
- Security scheme.

8.2 Hasil OpenAPI JSON

File openapi.json berhasil dibuat pada root project dan berisi seluruh endpoint WisataKu.

Endpoint yang terdapat pada file tersebut adalah:

/destinasi
/destinasi/{id}
/destinasi/{id}/ulasan

dengan method GET, POST, PATCH, dan DELETE sesuai kebutuhan API.

BAB IX

HASIL AKHIR PROJECT

9.1 Struktur Project

Setelah seluruh proses selesai, struktur project wisataku-api adalah:

```

wisataku-api/
├── src/
│   ├── destinasi/
│   │   ├── dto/
│   │   │   ├── create-destinasi.dto.ts
│   │   │   └── update-destinasi.dto.ts
│   │   ├── destinasi.controller.ts
│   │   ├── destinasi.module.ts
│   │   └── destinasi.controller.spec.ts
│   ├── app.controller.ts
│   ├── app.controller.spec.ts
│   ├── app.module.ts
│   ├── app.service.ts
│   └── main.ts
├── .gitignore
├── .prettierrc
├── nest-cli.json
├── package.json
├── package-lock.json
├── README.md
├── tsconfig.build.json
└── tsconfig.json

```

BAB X

HASIL AKHIR

Endpoint yang berhasil dibuat adalah:

No	Endpoint	Method	Keterangan
1	/destinasi	GET	Mendapatkan daftar destinasi
2	/destinasi	POST	Menambahkan destinasi
3	/destinasi/{id}	GET	Mendapatkan detail destinasi
4	/destinasi/{id}	PATCH	Memperbarui destinasi
5	/destinasi/{id}	DELETE	Menghapus destinasi

6	/destinasi/{id}/ulasan	GET	Mendapatkan ulasan
---	------------------------	-----	--------------------

Selain endpoint, project juga telah dilengkapi dengan:

1. CreateDestinasiDto.
2. UpdateDestinasiDto.
3. class-validator.
4. ValidationPipe.
5. Dokumentasi Swagger UI.
6. Dokumentasi OpenAPI dalam format JSON.
7. Pengujian endpoint.
8. Pengujian validasi data.